

ARTIFICIAL INTELLIGENCE ASSESSMENT – 2

REPORT

Course Code: CSA1704 – Artificial

Intelligence Submitted By

Name: RS Jahnavi

Register Number: 192371056

Department: Computer Science and Engineering

Acknowledgement

I express my sincere gratitude to my faculty members and the Department of Computer Science and Engineering for providing me with the opportunity to complete this Artificial Intelligence Assessment successfully. I also thank my friends and family members for their continuous encouragement and support throughout the completion of this work. Finally, I thank everyone who directly or indirectly contributed to the successful completion of this assessment.

Table of Contents

- 1. Introduction**
- 2. Objective**
- 3. Question 1 – Hospital Doctor Scheduling using Backtracking Search**
- 4. Question 2 – Robot Navigation using Breadth First Search**
- 5. Question 3 – Autonomous Rescue Robot using Online Search and Uniform Cost Search**
- 6. Python Implementation**
- 7. Program Outputs**
- 8. Conclusion**
- G. References**

Question 1 – Hospital Doctor Scheduling using Backtracking Search

Problem Statement

A hospital needs to assign three doctors (D1, D2, and D3) to three work shifts namely Morning, Afternoon, and Night. Each doctor must be assigned exactly one shift while satisfying the given scheduling constraints. The objective is to determine a valid schedule using the Backtracking Search Algorithm. This problem demonstrates the application of Constraint Satisfaction Problems (CSP) in Artificial Intelligence for solving scheduling and resource allocation problems efficiently.

Solution

The Hospital Doctor Scheduling problem is solved using the Backtracking Search algorithm. The algorithm assigns shifts to doctors one by one while checking all the given constraints. Whenever a constraint is violated, the algorithm returns to the previous assignment and tries another possible shift. This process continues until a valid schedule satisfying all the constraints is obtained.

State Representation

State = (D1, D2, D3)

Initial State = (-, -, -)

Goal State = (Morning, Afternoon, Night)

Constraints

- D1 cannot work in the Night shift.
- D2 must be assigned before D3.
- D3 cannot work in the Morning shift.
- Only one doctor can be assigned to each shift.

Architecture

Start

↓

Assign Shift

↓

Check Constraints

↓

Constraint Satisfied

↓

Yes → Assign Next Doctor

↓

No → Backtrack

↓

Goal State

Python Code:

```
doctors = ["D1", "D2", "D3"]
```

```
# List of available shifts
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
# Dictionary to store final assignment
```

```
assignment = {}
```

```
# -----
```

```
# Function to check whether assignment is valid
```

```
# -----
```

```
def is_safe(doctor, shift):
```

```
    # Constraint 1:
```

```
    # One shift should not be assigned twice
```

```
    if shift in assignment.values():
```

```
        return False
```

```
    # Constraint 2:
```

```
    # D1 cannot work Night shift
```

```

    if doctor == "D1" and shift == "Night":
        return False

# Constraint 3:
# D2 cannot work Morning shift
if doctor == "D2" and shift == "Morning":
    return False

return True
def backtrack(index):

    # Base condition
    if index == len(doctors):
        return True

    # Current doctor
    doctor = doctors[index]

    # Try every shift
    for shift in shifts:

        print(f"Trying {doctor} -> {shift}")

        # Check constraints
        if is_safe(doctor, shift):
            # Assign shift
            assignment[doctor] = shift

            print("Assigned:", assignment)

```

```

        # Recursive call
        if backtrack(index + 1):
            return True

        # Backtracking
        print("Backtracking from", doctor)

        del assignment[doctor]
        return False
    if backtrack(0):

        print("\nValid Schedule")

        for doctor, shift in assignment.items():
            print(doctor, "->", shift)

    else:
        print("No valid schedule found.")

```

Result

The Backtracking Search algorithm successfully generates a valid doctor schedule by satisfying all the given constraints.

Question 2 – Robot Navigation using Breadth First Search Problem

Statement

A robot is placed in a 5×5 grid environment where it has to move from the Start position to the Goal position while avoiding obstacles. The robot can move only in four directions namely Up, Down, Left, and Right. Each movement has a cost of one unit. The objective is to determine the shortest path using the Breadth First Search (BFS) algorithm. This problem demonstrates the application of AI search techniques in robot navigation and path planning.

Solution

The Robot Navigation problem is solved using the Breadth First Search (BFS) algorithm. The algorithm starts from the initial position and explores all neighbouring nodes level by level using a queue. Since every movement has the same cost, BFS always finds the shortest path from the start node to the goal node.

State Representation

State = (Current Position)

Initial State = Start Node (S) Goal

State = Goal Node (G)

Operators

- Move Up
- Move Down
- Move Left
- M o v e
Right

Architecture

Start Node

↓

Generate Successor Nodes

↓

Queue

↓

Expand Node

↓

Goal Test

↓

Goal Reached

Python Code:

```
from collections import deque
```

```
# 5x5 Grid
```

```
grid = [  
    ['S', '.', '.', '#', '.'],  
    ['.', '#', '.', '#', '.'],  
    ['.', '#', '.', '.', '.'],  
    ['.', '.', '#', '#', '.'],  
    ['#', '.', '.', 'G', '.']  
]
```

```
rows = 5
```

```
cols = 5
```

```
# Find Start and Goal positions
```

```
for i in range(rows):
```

```
    for j in range(cols):
```

```
        if grid[i][j] == 'S':
```

```
            start = (i, j)
```

```
        elif grid[i][j] == 'G':
```

```
            goal = (i, j)
```

```
# Possible movements
```

```
directions = [
```

```
    (-1, 0), # Up
```

```
    (1, 0), # Down
```

```
    (0, -1), # Left
```

```
    (0, 1) # Right
```

```
]
```

```
# BFS Function
```

```
def bfs(start, goal):
```

```
    queue = deque()
```

```
    queue.append((start, [start]))
```

```
    visited = set()
```

```
    visited.add(start)
```

```
    while queue:
```

```
        current, path = queue.popleft()
```

```
        print("Current Node:", current)
```

```
        print("Queue:", list(queue))
```

```
        if current == goal:
```

```
            return path
```

```
        for dx, dy in directions:
```

```
            x = current[0] + dx
```



```
y = current[1] + dy
```

```
if 0 <= x < rows and 0 <= y < cols:
```

```
    if grid[x][y] != '#' and (x, y) not in visited:
```

```
        visited.add((x, y))
```

```
        queue.append(((x, y), path + [(x, y)]))
```

```
return None
```

```
# Driver Code
```

```
path = bfs(start, goal)
```

```
if path:
```

```
    print("\nShortest Path:")
```

```
    print(path)
```

```
    print("\nTotal Cost =", len(path) - 1)
```

```
else:
```

```
    print("No Path Found")
```

Result

The Breadth First Search algorithm successfully identifies the shortest path from the start node to the goal node.

Question 3 –Autonomous Rescue Robot using Online Search and Uniform Cost Search

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected area to locate survivors safely. The robot operates in a partially observable environment containing blocked paths and risky zones. It continuously senses the environment, updates its knowledge, and

selects the least-cost path to reach the survivor. This problem demonstrates the use of Online Search Agents and Uniform Cost Search in intelligent decision-making.

Solution

The rescue robot is implemented as an Online Search Agent. It continuously collects information from the environment using sensors and updates its internal knowledge base.

The Uniform Cost Search (UCS) algorithm is used to identify the least-cost path by expanding the node with the minimum cumulative cost first. This enables the robot to reach the survivor efficiently while minimizing the total travel cost.

Environment

- Safe Area
- Blocked Area
- Risk Zone
- Survivor Location

Actions

- Move Forward
- Turn Left
- Turn Right
- Sense Environment
- Update Knowledge
- Rescue

Survivor

Architecture

Environment

↓

Sensors

↓

Knowledge Base

↓

Uniform Cost Search

↓

Decision Making

↓

Actuators

↓

Survivor Reached

Python Code:

```
import heapq
```

```
# 5x5 Grid
```

```
grid = [  
    ['S', '.', '.', '#', '.'],  
    ['.', 'R', '.', '#', '.'],  
    ['.', '#', '.', '.', '.'],  
    ['.', '.', 'R', '#', '.'],  
    ['#', '.', '.', 'G', '.']  
]
```

```
rows = 5
```

```
cols = 5
```

```
# Find Start and Goal
```

```
for i in range(rows):
```

```
    for j in range(cols):
```

```
        if grid[i][j] == 'S':
```

```
            start = (i, j)
```

```
        elif grid[i][j] == 'G':
```

```
            goal = (i, j)
```

```
# Possible moves
```

```
directions = [
```

```
    (-1, 0), # Up
```

```
    (1, 0), # Down
```

```
    (0, -1), # Left
```

```
    (0, 1) # Right
```

```
]
```

```
# Uniform Cost Search Function
```

```
def uniform_cost_search(start, goal):
```

```
    # Priority Queue
```

```
    pq = []
```

```
    # (cost, current_position, path)
```

```
    heapq.heappush(pq, (0, start, [start]))
```

```
    visited = {}
```

```
    while pq:
```

```
        cost, current, path = heapq.heappop(pq)
```

```
        print("Current Node:", current)
```

```
        print("Current Cost:", cost)
```

```
        if current == goal:
```

```
            return path, cost
```

```
        if current in visited and visited[current] <= cost:
```

```
    continue
```

```
visited[current] = cost
```

```
for dx, dy in directions:
```

```
    x = current[0] + dx
```

```
    y = current[1] + dy
```

```
    if 0 <= x < rows and 0 <= y < cols:
```

```
        if grid[x][y] != '#':
```

```
            # Normal move cost
```

```
            new_cost = cost + 1
```

```
            # Extra cost for risky zone
```

```
            if grid[x][y] == 'R':
```

```
                new_cost += 2
```

```
            heapq.heappush(
```

```
                pq,
```

```
                (new_cost, (x, y), path + [(x, y)])
```

```
            )
```

```
return None, None
```

```
# Driver Code
```

```
path, cost = uniform_cost_search(start, goal)
```

```
if path:
    print("\nMinimum Cost Path:")
    print(path)
    print("Total Cost =", cost)
else:
    print("No Path Found")
```

Result

The Online SearchAgent successfully reaches the survivor by selecting the minimum-cost path using the Uniform Cost Search algorithm.